

AI + CV

螃蟹精密尺寸测量系统

方案计划书

版本: v1.0

日期: 2026-06-22

平台: Orange Pi 5 Pro

SoC: Rockchip RK3585

NPU: 6 TOPS

摄像头: USB (V4L2)

基于 Orange Pi 5 Pro 的 AI + 传统 CV 融合螃蟹精密尺寸测量系统

目录 TOC

- 一、项目背景
- 二、方案概述
- 三、系统架构
- 四、核心模块设计
- 五、数据处理流水线
- 六、标定策略
- 七、项目目录结构（规划）
- 八、开发计划
- 九、技术栈
- 十、项目亮点
- 十一、注意事项

一、项目背景

本项目原为基于树莓派 4B 的螃蟹特征筛选视觉系统，现移植至香橙派 5 Pro (Rockchip RK3588)，并升级为 AI + 传统 CV 融合的精密测量系统。

硬件平台选型

- SoC: Rockchip RK3588
- NPU: 6 TOPS 算力, 支持 INT8 量化推理
- 摄像头: USB 摄像头 (V4L2 通用驱动)

目标: 构建一个基于端侧 AI 推理与机器视觉的螃蟹精密尺寸测量系统, 参加瑞心微 AI 视觉应用竞赛。

二、方案概述

本方案采用“AI 定位 + 传统 CV 精测”的双阶段架构。

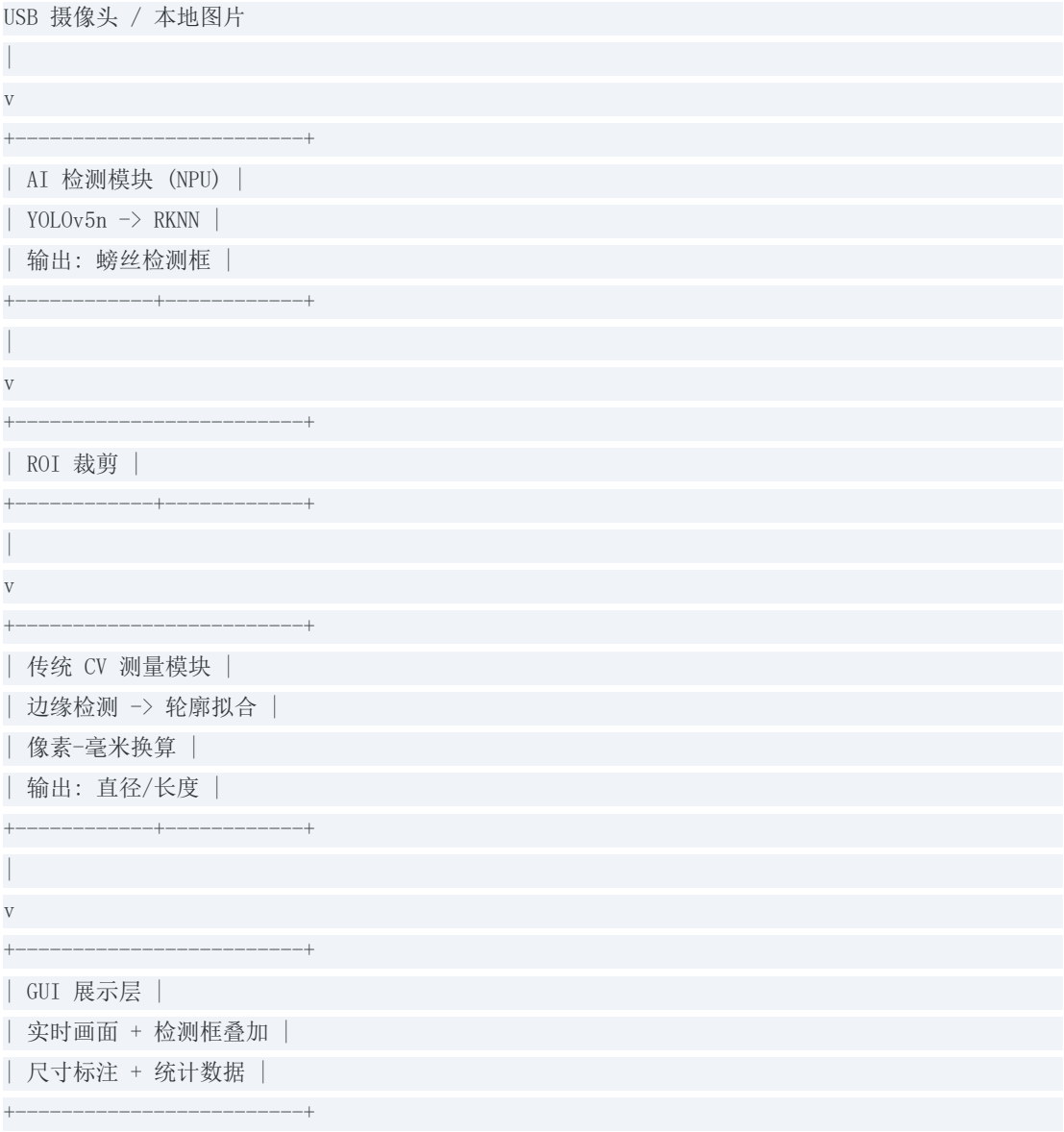
第一阶段，轻量 YOLO 模型在 RK3588 NPU 上实时推理，鲁棒地检测螃蟹在画面中的位置，输出检测框坐标。

第二阶段，在检测框 ROI 区域内，使用传统计算机视觉方法做亚像素级边缘检测与轮廓拟合，精确计算螃蟹的直径、长度等尺寸参数。

通过固定的像素-毫米标定系数换算为物理尺寸。

三、系统架构

系统采用分层模块化设计，从图像获取到结果展示形成完整闭环：



四、核心模块设计

4.1 AI 检测模块

项目	内容
文件	core/ai_detector.py (新建)
模型	YOLOv5n / YOLOv8n
部署格式	RKNN INT8 量化
推理后端	Rockchip NPU (RKNN Toolkit)
功能	检测螃蟹位置，输出检测框坐标

4.2 CV 测量模块

项目	内容
文件	core/measurement.py (新建)
方法	Canny 边缘检测 + 轮廓拟合
标定	一次性固定标定，系数存入 config.json
功能	在 ROI 内计算精密尺寸

4.3 协调控制器

文件：core/detector.py (改造)。协调 AI + CV 两阶段流水线，组合检测框、尺寸、判定结果为统一输出。

4.4 GUI 展示层

框架：PySide6 + QML。功能：实时画面、检测框叠加、尺寸标注、检测统计、参数配置面板。

五、数据处理流水线

Step 1: 帧获取

USB 摄像头实时采集 (cv2.VideoCapture)，或本地图片模拟 (开发阶段)。

Step 2: AI 推理

YOLO 模型在 NPU 上推理，返回检测框坐标和置信度。

Step 3: ROI 裁剪

根据检测框裁剪出螃蟹区域。

Step 4: CV 精密测量

边缘检测与轮廓拟合，像素尺寸 $\times$ 标定系数 = 物理尺寸，输出直径、长度等参数。

Step 5: 结果展示

GUI 叠加显示检测框和尺寸，更新统计面板。

六、标定策略

采用一次性固定标定方案。设备搭建完成后，将一枚已知直径的硬币或标准件放置在拍摄区域，系统自动识别并计算像素-毫米转换系数。

标定参数存储于 `config.json` 的 `calibration` 字段。后续日常使用无需重复标定，亦无需在每张图像中放置参照物。

七、项目目录结构（规划）

```
OPI_Feature_System_focus_vision/
|-- main.py CLI 主入口
|-- main_gui.py GUI 主入口 (PySide6+QML)
|-- main_gui_pyqt6.py 旧版 PyQt6 入口
|-- config.json 配置文件
|-- requirements.txt 依赖清单
|-- README.md 项目文档
|-- AI+CV.pdf 本方案文档
|
|-- core/
|   |-- __init__.py
|   |-- detector.py [改造] 协调 AI+CV 流程
|   |-- ai_detector.py [新建] AI 检测模块
|   +-- measurement.py [新建] CV 精密测量模块
|
|-- hardware/
|   |-- __init__.py
|   +-- hardware_stub.py 硬件控制接口
|
|-- ui/
|   |-- __init__.py
|   |-- main_window.py 旧版 PyQt6 界面
|   |-- vision_backend.py PySide6 后端隔离层
|   +-- VisionUI.qml QML 深色玻璃质感界面
|
|-- models/
|   |-- best.rknn [新建] RKNN 量化模型
|   +-- best.onnx [新建] ONNX 模型
|
|-- samples/ 测试图片
+-- scripts/
+-- calibrate.py [新建] 标定工具脚本
```

八、开发计划

阶段一：数据准备

拍摄 50-100 张俯视螃蟹照片（含参照物）；使用 LabelImg 标注螃蟹位置；数据增强扩充训练集。

阶段二：PC 端算法开发

调通传统 CV 测量模块；训练 YOLO 检测模型；实现 AI + CV 融合流水线；完善 GUI 界面。

阶段三：模型部署

ONNX 导出；RKNN Toolkit 2 量化转换；在 Orange Pi 5 Pro 上验证 NPU 推理。

阶段四：系统集成

接入 USB 摄像头实时画面；完整流水线联调；FPS 性能优化。

阶段五：硬件分拣（远期可选）

步进电机、舵机控制实现；AI 检测结果 -> 硬件动作联动。

九、技术栈

分类	技术栈
编程语言	Python 3.8+
AI 框架	Ultralytics (YOLO), PyTorch, ONNX
NPU 工具链	RKNN Toolkit 2
传统视觉	OpenCV, NumPy, SciPy
GUI 框架	PySide6 + QML (新版) / PyQt6 (备用)
硬件控制	OPi.GPIO / python3-gpiod
标定与测试	config.json, OpenCV 标定模块

十、项目亮点

1. 端侧 AI 推理

YOLO 模型在 RK3585 NPU 上实时推理，充分体现瑞心微平台的 AI 加速能力。

2. AI + CV 融合架构

取 AI 的鲁棒性与 CV 的精度之长，不是简单的替代关系，而是互补协作。

3. 完整的视觉-控制闭环

从图像采集到 AI 检测再到精确测量，最终衔接硬件分拣控制，形成完整方案。

4. 工程化设计

模块解耦、参数可配置、标定策略完善，体现了扎实的工程落地能力。

5. 平台适配

基于 Orange Pi 5 Pro 开发，完全契合瑞心微端侧 AI 竞赛的平台要求。

十一、注意事项

- YOLO 训练数据和标注质量直接影响检测效果。
- 拍摄距离和角度需尽量与最终使用场景一致。
- 像素-毫米标定需在设备固定后进行。
- USB 摄像头选择建议： $\geq 1080p$ ，手动对焦优先。
- RKNN 转换需在 x86 主机上进行，橙派仅加载推理。

— 文档结束 —